

A Nested Basket Model for the dunnhumby Complete Journey Data

Specification, inference, sampling, and an empirical study

Abstract

We develop a model of supermarket baskets designed to answer price counterfactuals and to generate synthetic shopping data. The model separates three decisions that a single-choice model conflates — whether a household buys from a category, which item it then picks, and how much of it — and estimates the coupling between them rather than assuming it. Two commitments organise the work. First, every component is introduced to answer a measurement taken on the data beforehand, and Section 3 reports those measurements. Second, every claim is a number on held-out weeks, and where a number is not comparable across settings we say so. The fitted model beats the strongest simple baseline by +0.342 nats, its price coefficient retains 0.0% of its value when the price panel is scrambled before fitting, and baskets it generates carry 87% of the real price elasticity. Along the way we report a negative result and its resolution: eleven changes to the model’s interaction term failed to make generated baskets reproduce which items co-occur, and a change to the *negative sampling distribution* succeeded.

Contents

1	Introduction	2
1.1	Main idea	2
1.2	Main results	3
1.3	Relation to prior work	3
2	Data	3
2.1	Notation	4
2.2	How the price series is built	4
3	What the data says	5
3.1	Demand responds to price	6
3.2	A price cut moves three margins, and they can be measured separately	6
3.3	Promotions: an event study	6
3.4	Households differ, and by how much can be measured	7
3.5	Stores are not interchangeable	8
3.6	Base rates the model must reproduce	8
4	The model	8
4.1	The shared quantity: how appealing item j is	8
4.2	The four answers	9
4.3	The term that ties them together, and what κ means	11
4.4	Letting items in a cart influence each other	12
4.5	The probability of a whole basket	13
4.6	What is fitted, and the three ways it differs from Eq. 22	13

5	Inference	14
5.1	Objective	14
5.2	There is no negative sampling	15
5.3	The inclusive value is estimated, not summed	15
5.4	Category sampling for the incidence head	15
5.5	Computational cost	16
6	Sampling	16
7	From measurement to model	17
8	Empirical study	17
8.1	Protocol, and what is comparable	18
8.2	Do the equations match the code?	18
8.3	Ablations	18
8.4	Against learned baselines	19
8.5	Is the price response identified?	19
8.6	Where a price cut goes	20
8.7	Generation	20
8.8	What the representations learned	21
9	Related work	22
10	Discussion	22

1 Introduction

1.1 Main idea

A retailer cutting one price wants to know what happens to the whole basket. Three things can happen and they are economically distinct: households that were not going to buy the category at all now do (*incidence*); households already buying the category switch to the discounted item (*allocation*); and households buying that item take more of it (*quantity*). A model that only ranks items cannot tell these apart, and the difference between them decides whether a promotion grows a category or merely cannibalises it.

Our model gives each of these its own likelihood term, sharing one item utility u_{ijt} . The coupling between incidence and allocation is a single estimated parameter per category, κ_c , with a standard nested-logit reading: at $\kappa = 1$ category volume is whatever the items sum to, at $\kappa = 0$ a price cut only moves share, and above 1 the category expands more than proportionally.

A second idea runs through the paper. The purpose of the model is partly to *generate* data — synthetic baskets on which a pricing or couponing policy could be learned — and generation is a far more demanding test than prediction. A model can score well at “which item completes this basket” while producing synthetic baskets that look nothing like real ones. We therefore evaluate the generator on five model-free, distributional checks (Section 8.7), and most of what we learned came from those rather than from the likelihood.

1.2 Main results

1. **The three margins are separately identified and they are not equal.** A price cut splits 79% allocation, 12% incidence, 9% quantity (Section 8.6). The quantity margin is invisible to a binary-purchase model.
2. **The price response survives a structural placebo.** Refitting on a scrambled price panel retains 0.0% of the coefficient while leaving κ and the incidence likelihood untouched (Section 8.5). This rules out a spurious constant; it does not identify the response against promotion timing, and we do not claim it does.
3. **It beats the published alternative on the published alternative’s own metric.** Against a reimplemented SHOPPER, +0.320 nats within category and +0.310 on the full catalogue (Section 8.4).
4. **The item conditional is exact.** Normalising over the purchased item’s category rather than the catalogue removes negative sampling entirely (Section 5.2), so no reported likelihood depends on a proposal distribution.
5. **Product embeddings carry real structure; household embeddings carry none.** Sub-commodity purity at $27.8\times$ chance with two nulls at chance, against gradient-boosted demographic probes that fail to beat the majority class on any target (Section 8.8).
6. **Matching means is not matching distributions.** A discriminator separates real from generated baskets at AUC 0.77, and the generator is 9% short on basket size for reasons we have not established (Section 8.7).
7. **Two negative results and two reversals are reported rather than buried.** Persistent contrastive divergence and an explicit habit feature both moved generation by nothing; a temperature conclusion and a coupon-targeting conclusion both reversed once every number was recomputed on a single fitted model (Sections 8.7, 8.8).

1.3 Relation to prior work

The model is a descendant of SHOPPER [1] and of the nested factorization of Donnelly et al. [2]. From SHOPPER we take the item-attribute factorisation, the per-household price sensitivity as a low-rank bilinear form, and week effects as a seasonality control. We depart from it in four ways: we add an explicit category-incidence stage with an estimated nesting coefficient, we model units rather than binary purchase, we condition on the store, and we replace its sampled softmax with an exact one over the purchased item’s category. Section 9 discusses this in detail.

2 Data

The dunnhumby “Complete Journey” dataset: two years of transactions from one grocery chain. Construction is in `scripts/22_basket_data.py`.

households N	2,066	(basket, item) rows	1,566,063
items J	5,455	baskets	199,345
categories C	188	train / val / test rows	1,228,695 / 134,665 / 202,703
sub-commodities S	758	units > 1	22.3% of rows, 42.6% of units
stores	115	assortment density	57.2%
days D	712	median repurchase gap	28 days

Filters. An item needs at least 100 purchase lines, which is statistical support for its own parameters; a household needs 20–300 trips. Nothing is dropped to protect a modelling assumption — in particular no basket is discarded for containing two items from one category, which is the filter that one-item-per-category models require and which removes most of the data.

Splits are temporal. Train is week < 83, validation 83–90, test ≥ 91 . A model whose purpose is to answer “what happens next week” must be scored on later weeks; a random split would let it see each household’s future. Held-out rows whose item or household never appears in training are dropped rather than scored as a cold start.

Prices. The model sees a within-item log price deviation,

$$\Delta \log p_{jst} = \Delta \log p_{jt} + d_{jsw}, \quad (1)$$

built in six steps set out in Section 2.2. Deviating within item means the item’s price *level* is absorbed by its own intercept, so what identifies the price response is an item moving against its own normal price rather than expensive items differing from cheap ones.

2.1 Notation

symbol	meaning	symbol	meaning
i	household, 1.. N	N	2,066 households
j, k, m	item, 1.. J	J	5,455 items
c	category, 1.. C	C	188 categories
t	day, 1.. D	D	712 days in the panel
w	week-of-year, 1..52	s	store, 1..115
S	the set of items in a basket	n	$ S $, basket size
B	the set of categories bought on a trip	n_c	items of category c the store carries
R	purchase rows in a training batch	C_r	candidate set for row r
sub(j)	item j ’s sub-commodity, of 758	τ_{ijt}	days since i last bought sub(j)
π_j	within-category choice probability	λ	a Poisson rate

2.2 How the price series is built

The price entering Eq. 1 is the end of a chain, and each step is a choice.

Step 1: which price. A dunnhumby transaction line records four fields we need: **SALES_VALUE** (what the retailer banked), **QUANTITY**, **COUPON_MATCH_DISC** (the retailer’s match of a manufacturer coupon), and **RETAIL_DISC** (the loyalty-card discount). Three different prices can be reconstructed

and they answer different questions. We use the **loyalty price**, what a card holder faces at the shelf:

$$\text{price}_\ell = \frac{\text{SALES_VALUE} - \text{COUPON_MATCH_DISC}}{\text{QUANTITY}}. \quad (2)$$

Subtracting the coupon match removes the part of the receipt the manufacturer refunded, which is not a price the shopper responded to. We do *not* use the paid price (which adds `COUPON_DISC` back), because two shoppers at the same shelf with different coupons would then face different “prices” for the same item on the same day, and the model has one price per (item, day).

Step 2: one number per item-day. price_{jt} is the *median* loyalty price over all purchase lines of item j on day t . A median rather than a mean because a single mis-keyed quantity produces a wild per-unit value.

Step 3: fill the gaps. Only 24.7% of the $J \times D$ item-day grid is directly observed. Missing days are filled by carrying the last observed price forward, then the first observed price backward for days before an item’s first sale. This assumes a posted price persists until it is next seen to change, which is what a shelf tag does.

Step 4: log and deviate.

$$\log p_{jt} = \log \max(\text{price}_{jt}, 10^{-3}), \quad \Delta \log p_{jt} = \log p_{jt} - \frac{1}{D} \sum_{t'=1}^D \log p_{jt'}, \quad (3)$$

where the mean runs over **all** $D = 712$ **days of the panel**, filled days included, so it is the item’s average posted price over the two years rather than an average over the days it happened to sell.

Step 5: the store deviation. For each (item, store, week) actually observed,

$$d_{jsw} = \log \tilde{p}_{jsw} - \log \tilde{p}_{jw}, \quad (4)$$

where $\tilde{p}_{jsw}$ is the median loyalty price of item j at store s in week w and $\tilde{p}_{jw}$ the median over the whole chain that week, kept only where $|d_{jsw}| > 10^{-6}$. There are 244,880 such cells, 0.53% of the (item-week $\times$ store) grid, so the overwhelming majority of lookups return $d_{jsw} = 0$ and fall back to the chain price. That is deliberate: imputing a store price from nothing would invent variation the data does not contain.

Step 6: availability. $\text{carried}[j, s]$ is true if store s sold item j *at any point* in the two years. The threshold is one sale, not three: an item a store ever sold was by definition available there, and a threshold of three marked genuine purchases as unavailable, which set the inclusive value to -10^9 for categories a household demonstrably bought from. The mask covers 57.2% of the item $\times$ store grid. `dunnhumby` has no stock-out feed, so this is a proxy and remains one.

3 What the data says

Every measurement in this section was taken before any model was fitted, using counts and least squares only. Each subsection ends with the design consequence, and Section 7 collects those consequences into a table. The code is `scripts/29_demand_eda.py` and `scripts/30_household_eda.py`.

3.1 Demand responds to price

Form a panel with one row per (item, week). Let b_{jw} be purchase lines and T_w the number of baskets chain-wide that week; put $y_{jw} = \log \frac{b_{jw} + 0.5}{T_w}$, where the 0.5 keeps zero-purchase weeks in the panel, and let x_{jw} be the mean log price. The elasticity is a within-item least-squares slope, i.e. after removing each item’s own mean from both sides:

$$\hat{\varepsilon} = \frac{\sum_{j,w} (x_{jw} - \bar{x}_j)(y_{jw} - \bar{y}_j)}{\sum_{j,w} (x_{jw} - \bar{x}_j)^2}, \quad 556,410 \text{ item-weeks.} \quad (5)$$

outcome	elasticity	interpretation
log buyers per trip	−0.792	whether a household buys the item at all
log units per trip	−0.945	total units moved
log units per buyer	−0.235	how much a buyer takes, given that they buy

Of the total units response, 25% comes from existing buyers taking more. **Consequence:** a binary-purchase model discards a quarter of the price response, so quantity needs its own likelihood term.

3.2 A price cut moves three margins, and they can be measured separately

Units from a category factor exactly,

$$\frac{\text{units}}{\text{basket}} = \underbrace{\frac{\text{visits}}{\text{baskets}}}_{\text{incidence}} \cdot \underbrace{\frac{\text{lines}}{\text{visit}}}_{\text{breadth}} \cdot \underbrace{\frac{\text{units}}{\text{line}}}_{\text{depth}}, \quad (6)$$

so in logs the three elasticities must sum to the total. Estimating each by Eq. 5 on a (category, week) panel of 15,309 cells:

margin	elasticity
incidence	−0.675
breadth	−0.092
depth	−0.158
sum	−0.925 against −0.945 from Eq. 5 on the item panel

The two agree to 0.021 across different panels, different units of analysis and no shared code. About 73% of the entire units response is households deciding to enter a category at all. **Consequence:** incidence is the dominant margin and must be modelled explicitly, not folded into item choice.

3.3 Promotions: an event study

Take every item-week where log price fell by at least 0.15 against the previous week (37,132 events over 4,712 items) and normalise demand by the item’s own mean. Events are not independent: 57.7% of consecutive cuts for the same item fall less than seven weeks apart, so we also report the 20,725 *isolated* events with no other cut within ± 3 weeks. Neither 1.0 nor the pre-period is a valid baseline — the former averages over the promotions themselves, the latter is selected to be

expensive by the event definition — so we use a *quiet week*, an item-week more than three weeks from any cut of that item. There are 368,173, running at 0.896 of the item’s mean demand at essentially its normal price.

weeks from cut	demand	price (log dev.)	demand ÷ quiet
−3	0.925	+0.050	1.03
−1	0.870	+0.123	0.97
0	2.136	−0.194	2.38
+1	1.337	−0.139	1.49
+3	1.065	−0.003	1.19
quiet	0.896	+0.011	1.00

Three readings. *Demand does not ramp before a cut*: the pre-period sits within 3% of quiet, so the spike is not partly its own cause — which is what licenses reading the price coefficient causally, and is tested directly in Section 8.5. *The promotion week is much steeper than elasticity*: from −1 to 0, price moves −0.317 and demand +0.898 in logs, an implied slope of −2.83, which is 3.4× the −0.792 measured on ordinary variation — a promoted price arrives with display and feature that the price variable does not carry. *Most of the tail is the discount still running*: at +1 price is still 0.139 below normal, and by +3, when price has returned, demand is 1.19 of quiet. No offset dips below quiet, so no stockpiling is visible within three weeks.

3.4 Households differ, and by how much can be measured

Taste. Split each household’s purchases in half by date, count purchases per category in each half to get two vectors in $\mathbb{R}^{188}$, scale each to unit length, and take the dot product. Against *all* 4,266,290 cross-household pairs:

own two halves	0.7838
two different households	0.4710
ratio	1.66×

A stranger already scores 0.4710 because everyone buys milk and bread; the gap is what is personal. **Consequence:** a per-household taste vector θ_i is justified.

Price sensitivity is real but poorly measured. Regress log units on log price per household, within (household, item), on pairs bought three or more times. Across 1,640 households the median is −0.169 with p10–p90 −0.427—−0.016. But spread alone proves nothing — noisy estimates are spread out even when the truth is identical — and measuring each household twice on disjoint halves gives a split-half correlation of only +0.236 on 1,374 households.

The reason is visible in the estimator, which divides by the price variation that household actually saw: 18.6% of qualifying rows come from a (household, item) pair whose price never moved. Splitting households by how much movement they had:

	spread of slopes (sd)	share positive
little price movement	0.226	11.5%
much price movement	0.129	3.5%

Where prices moved, estimates are tighter and almost all negative. **Consequence:** per-household price sensitivity is justified but must be low-rank rather than free, and a targeting policy should weight by how much is known about each household.

3.5 Stores are not interchangeable

15.8% of store-item-weeks sit more than a cent from the chain price, and the median store ever sold only 63% of the catalogue. **Consequence:** scoring an item a store never stocked as a *rejected* alternative is a specification error; it was not available to reject.

3.6 Base rates the model must reproduce

Categories per basket 6.12 of 188 (3.25%); items per basket 7.86; units per basket 10.64; distinct items per purchased category 1.284. These are the quantities Section 8.7 holds the generator to.

4 The model

A trip is one household i shopping at store s on day t , in week-of-year w . The model answers four questions about that trip, in this order:

1. For each of the C categories — does the household buy from it at all?
2. For a category it buys from — how many *different* products does it take?
3. Which products?
4. How many units of each?

Section 4.1 builds one quantity that all four answers share. Section 4.2 gives the four answers. Section 4.3 explains the one term that ties question 1 to questions 2–4, and what its coefficient means. Section 4.4 covers the term that lets items in a basket influence each other. Section 4.5 multiplies the four answers into the probability of a whole basket and says how the training objective differs from it.

4.1 The shared quantity: how appealing item j is

Everything rests on one number per (household, item, trip), which we call the item’s utility:

$$\begin{aligned}
 u_{ijt} = & \underbrace{\lambda_j}_{\text{popular?}} + \underbrace{\theta_i^\top \alpha_j}_{\text{their kind of thing?}} + \underbrace{\rho_j^\top \bar{\alpha}_j(S)}_{\text{fits the cart?}} - \underbrace{(\gamma_i^\top \beta_j) \Delta \log p_{jst}}_{\text{expensive today?}} \\
 & + \underbrace{\eta_j^\top x_{ijt}}_{\text{due a repurchase?}} + \underbrace{\mu_j^\top \delta_w}_{\text{in season?}} + \underbrace{\zeta_j^\top \xi_s}_{\text{this store's kind of thing?}} + \underbrace{w_j^{\text{loy}} \omega_{ij} + w_j^{\text{frq}} f_{ij}}_{\text{their usual?}}
 \end{aligned} \tag{7}$$

Each term is a dot product of two learned vectors, which is a compact way of writing a large table of interactions with few numbers. $\theta_i^\top \alpha_j$ is an example: rather than storing one taste value for every household–item pair ($N \times J$ numbers), each household gets a vector θ_i , each item a vector α_j , and their dot product supplies the pair. The same device gives every household its own price sensitivity $\gamma_i^\top \beta_j$ instead of one shared slope.

The price input. $\Delta \log p_{jst}$ is how far today’s price sits from that item’s own normal price, in logs (Section 2.2). It enters with a minus sign, so a *positive* $\gamma_i^\top \beta_j$ means the household buys less when the price is above normal.

The memory input. x_{ijt} describes how long it has been since this household last bought something of this kind. Let τ be the days since household i last bought item j ’s sub-commodity; then

$$x_{ijt} = [\mathbb{1}\{\text{never before}\}, \quad e^{-\tau/7}, \quad e^{-\tau/g_{\text{sub}(j)}}, \quad \log(1 + \tau)/\log 100]^\top, \quad (8)$$

with the last three set to zero if there is no earlier purchase. The three decays let the model express “bought it yesterday”, “about due”, and “long gone” separately, and $g_{\text{sub}(j)}$ is that sub-commodity’s median repurchase gap so that “due” means a week for milk and months for shampoo.

The habit input. $\theta_i^\top \alpha_j$ is a rank- K bilinear form standing in for an $N \times J$ table, so it can say “this household likes this kind of product” but not “this household buys *this* product and no other”. Two scalars supply the difference. With $n_{ij}^\mathcal{T}$ the number of training trips on which household i bought item j ,

$$\omega_{ij} = \frac{n_{ij}^\mathcal{T}}{\max(1, \sum_{j' \in \mathcal{C}(c(j))} n_{ij'}^\mathcal{T})}, \quad f_{ij} = \frac{\log(1 + n_{ij}^\mathcal{T})}{\log 100}. \quad (9)$$

ω_{ij} is the household’s within-category share of item j — exactly the quantity the item conditional normalises, so $\sum_{j \in \mathcal{C}(c)} \omega_{ij} = 1$ for any category the household has bought from — and f_{ij} the absolute strength of the habit. Both are computed on the training window only, so they are past lookups rather than leakage, and both are fixed *data*: only the per-item loadings w_j^{loy} , w_j^{frq} are estimated. Being frozen at the training window, they do not update inside a generated rollout, which is one reason Section 6 produces a sampler and not a simulator.

4.2 The four answers

Write $\mathcal{C}_s(c)$ for the items of category c that store s carries. All four answers are read off u_{ijt} .

1. Is category c bought? A coin flip whose bias depends on the household, the season of its last purchase, and how good the category looks today:

$$y_{ct} \sim \text{Bernoulli}(\sigma(\eta_{ict})), \quad \eta_{ict} = \underbrace{c_c^0 + c_i^{u^\top} c_c^c + c_c^{s^\top} x_{ict}}_{\text{household and timing}} + \underbrace{\kappa_c (IV_{ict} - \overline{IV}_c)}_{\text{how good the items are}}, \quad (10)$$

with $\sigma(x) = 1/(1 + e^{-x})$ turning the score into a probability. Term by term:

symbol	shape	what it is
y_{ct}	scalar $\in \{0, 1\}$	1 if the household buys anything from category c on this trip
c_c^0	one per category	how often c is bought at all: bread often, motor oil rarely
c_i^u	K per household	what kinds of category this household leans toward
c_c^c	K per category	where category c sits among those kinds
$c_i^{u\top} c_c^c$	scalar	their match: this household’s appetite for this category
x_{ict}	4 numbers	the recency basis of Section 4.1, for this category
c_c^s	4 per category	how strongly recency matters for this category
κ_c	one per category	how much the items’ appeal feeds into buying the category
IV_{ict}	scalar	how good the category’s items look today (Section 4.3)
$\overline{IV}_c$	one per category	a constant subtracted from IV , fixed once during fitting (Section 5.3)

So the first three terms answer “would this household buy this category on a normal day?” and the fourth adjusts for what the category is worth *today*, given its prices and what the store has.

What x_{ict} actually contains, and why it is wrong. The recency basis of Section 4.1 is defined per *sub-commodity*. A category contains several, so a category-level recency has to be constructed somehow. The code does this (`27_nested_basket.py:425`):

1. take the item in category c with the lowest `item_id`, which is the lowest `PRODUCT_ID` — an arbitrary ordering with no meaning;
2. read off that one item’s sub-commodity;
3. set τ = days since household i last bought *anything in that sub-commodity*;
4. return $x_{ict} = [\mathbb{1}\{\text{never}\}, e^{-\tau/7}, e^{-\tau/g_{\text{sub}}}, \log(1 + \tau)/\log 100]^\top$.

So it is not a category-level quantity at all. It is the recency of one arbitrarily chosen sub-commodity, standing in for the category. Concretely:

category	items	subs	sub-commodity actually used	% of items
BAKED BREAD/BUNS/ROLLS	141	14	HOT DOG BUNS	8.5%
BAG SNACKS	153	12	PRETZELS	12.4%
CHEESE	176	11	NATURAL CHEESE CHUNKS	19.9%
FRZN MEAT/MEAT DINNERS	201	7	PREMIUM ENTREES	21.9%
SOFT DRINKS	225	14	2 LITER BOTTLE, CARBONATED	30.7%
YOGURT	148	2	YOGURT NOT MULTI-PACKS	76.4%

For bread, the model’s answer to “is this household due to buy bread?” is “have they recently bought hot dog buns?”. Across all 188 categories the median chosen sub-commodity covers 50% of its category’s items, and in 21% of categories it covers under 20%.

This is a defect, not a modelling choice, and we record it as one (Section 10). Its effect is bounded by what the term is worth: $c_c^{s\top} x_{ict}$ sits inside the incidence score alongside the household and inclusive-value terms, and the state ablation in Section 8.3 bounds the whole recency mechanism — item-level and category-level together — at 0.071 nats. A correct version would use the days since the household last bought *any* item in the category, which is a change to the data build rather than the model; we have not made it, so every number in this paper uses the arbitrary-sub-commodity version.

2. How many different products, given it is bought?

$$k_{ct} - 1 \sim \text{Poisson}(e^{z_{ict}^b}), \quad z_{ict}^b = b_c^0 + b_i^u - b_c^p \overline{\Delta \log p_{ct}}, \quad (11)$$

Subtracting 1 and adding it back means $k \geq 1$: a bought category contains at least one product, and the Poisson only models how many *extra* ones.

symbol	shape	what it is
k_{ct}	integer ≥ 1	how many different products are taken from category c
b_c^0	one per category	the typical breadth of this category — one yoghurt or four
b_i^u	one per household	whether this household buys broadly or narrowly, overall
b_c^p	one per category	how much a category-wide promotion widens the selection
$\overline{\Delta \log p_{ct}}$	scalar	average price deviation over the category's stocked items

The minus sign on b_c^p means a positive b_c^p widens the basket when prices fall.

3. Which products? The k_{ct} products are drawn, without replacement, in proportion to e^u among the items the store carries — so an item twice as appealing is twice as likely, and an item the store does not stock cannot be chosen at all:

$$P(j) = \frac{e^{u_{ijt}}}{\sum_{j' \in \mathcal{C}_s(c)} e^{u_{ij't}}}, \quad j \in \mathcal{C}_s(c). \quad (12)$$

4. How many units of each chosen product?

$$q_{jt} - 1 \sim \text{Poisson}(e^{z_{ijt}}), \quad z_{ijt} = q_j^0 - (\gamma_i^{q\top} \beta_j^q) \Delta \log p_{jst} + \eta_j^{q\top} x_{ijt}, \quad (13)$$

Again $q \geq 1$: a chosen product is bought at least once, and the Poisson models the extras.

symbol	shape	what it is
q_{jt}	integer ≥ 1	units of product j
q_j^0	one per item	how many people usually take: one ketchup, six yoghurts
γ_i^q	K_p per household	willingness to stock up when something is cheap
β_j^q	K_p per item	how stockpile-able this item is
η_j^q	4 per item	how time since last purchase changes the amount

Note the price coefficient here, $\gamma_i^{q\top} \beta_j^q$, is *separate* from the one in Eq. 7. That is deliberate: the data says whether a household buys an item and how much of it respond to price at different rates (Section 3.1), and sharing one coefficient would assume away the difference.

4.3 The term that ties them together, and what κ means

The four answers would be independent were it not for one term in Eq. 10. The *inclusive value*

$$IV_{ict} = \log \sum_{j \in \mathcal{C}_s(c)} e^{u_{ijt}} \quad (14)$$

collapses a whole category into one number: how appealing its items are, right now, at this store, at today's prices. If everything in the category goes on offer, every u_{ijt} rises and so does IV . The coefficient κ_c decides how much of that reaches the decision to buy the category at all.

This is the model's answer to the retailer's question. A price cut on one item raises IV a little; κ_c says whether that turns into new category buyers or merely reshuffles Eq. 12:

$\kappa_c = 0$	incidence ignores the items completely. A price cut moves share inside the category and nothing else — pure cannibalisation.
$\kappa_c = 1$	the reference point (see below): the category grows exactly as much as its items gained, and no more.
$\kappa_c > 1$	the category grows more than its items gained — the promotion pulls in trips.

Why $\kappa = 1$ is the reference point. Suppose instead of a coin flip we modelled the total units a household takes from category c as a count $Q \sim \text{Poisson}(e^{a_{ic} + \kappa_c IV_{ict}})$, split across the category's items in proportion to e^u . A Poisson total split by fixed proportions gives independent Poisson counts per item, with rates $e^{a_{ic} + \kappa_c IV} \cdot e^{u_{ijt} - IV}$, because the proportion for item j is exactly $e^{u_{ijt} - IV}$ by Eq. 14. That is

$$q_{ijt} \sim \text{Poisson}(e^{a_{ic} + (\kappa_c - 1)IV_{ict} + u_{ijt}}), \quad (15)$$

and at $\kappa_c = 1$ the inclusive value disappears: each item's rate depends only on its own u , so the category's total is whatever its items happen to add up to. That is the sense in which $\kappa = 1$ means “no expansion”, and it is why we parameterise $\kappa_c = \text{softplus}(\kappa_c^{\text{raw}})$ starting at exactly 1.0: the data must push it off the neutral point in one direction or the other.

Eq. 15 is a device for reading κ , not a second model. The fitted model is Eqs. 10–13, in which the inclusive value appears once, in the incidence term.

4.4 Letting items in a cart influence each other

The remaining term in Eq. 7 is $\rho_j^\top \bar{\alpha}_j(S)$, where S is the set of items in the basket. For item j in a basket of n items, the context is

$$\bar{\alpha}_j(S) = \frac{1}{n-1} \left(\sum_{k \in S} \alpha_k - \alpha_j \right), \quad \bar{\alpha}_j(S) := \mathbf{0} \text{ when } n = 1. \quad (16)$$

Why this is an average over the other items. Item j is itself in the basket, so $\sum_{k \in S} \alpha_k$ already includes α_j ; subtracting it leaves the sum over everything else,

$$\sum_{k \in S} \alpha_k - \alpha_j = \sum_{k \in S, k \neq j} \alpha_k, \quad \text{so} \quad \bar{\alpha}_j(S) = \frac{1}{n-1} \sum_{k \in S, k \neq j} \alpha_k. \quad (17)$$

S has n items and one of them is j , so there are $n-1$ others; dividing by $n-1$ makes this their average. When $n = 1$ there are no others, $n-1 = 0$, and the term is set to zero.

Why the interaction is an average of pairwise affinities. A dot product is linear, so the constant comes out and $\rho_j^\top$ goes inside the sum:

$$\rho_j^\top \bar{\alpha}_j(S) = \frac{1}{n-1} \sum_{k \in S, k \neq j} \rho_j^\top \alpha_k. \quad (18)$$

Each $\rho_j^\top \alpha_k$ is one number saying how much having k in the cart helps or hurts j : positive means they go together, negative means they compete. The term is the average of those over everything else in the cart. For a cart $\{j, k, l, m\}$ it is $\frac{1}{3}(\rho_j^\top \alpha_k + \rho_j^\top \alpha_l + \rho_j^\top \alpha_m)$.

Eq. 16 is what the code computes, because the basket sum is accumulated once and each item's context follows from one subtraction. The data records no order within a receipt, so S is a set: the context is the whole cart, never a prefix.

Tied or untied. Setting $\rho_j = \alpha_j$ makes the interaction $\alpha_j^\top \alpha_k$, which is symmetric and largest when two items are *alike*. That forces co-purchase structure into α , which is what makes the embedding recover product categories (Section 8.8), but it cannot say that two *different* items belong together. Leaving ρ free allows that. The tied form is nevertheless the default, and not only by preference: Eq. 20 requires the pairwise interaction to be *symmetric*, and an untied $\rho_j^\top \alpha_k$ is not, so no potential function exists and $P(S \mid \mathcal{K})$ would not be defined. Symmetry can be had without tying — $\frac{1}{2}(\alpha_j^\top \rho_k + \rho_j^\top \alpha_k)$ admits a potential — so tying is a stronger restriction than the theory needs, and it costs the ability to represent an arbitrary symmetric co-purchase structure: $W = AA^\top$ is positive semidefinite of rank at most K .

What distribution over baskets this implies. Write b_j for every part of Eq. 7 that does not involve other basket items. For baskets of a fixed size n , define a score for the whole basket,

$$E(S) = \sum_{j \in S} b_j + \frac{1}{n-1} \sum_{\{j,k\} \subset S} \alpha_j^\top \alpha_k, \quad (19)$$

where $\{j, k\} \subset S$ runs over each unordered pair once, and let higher-scoring baskets be more likely, $P(S) \propto e^{E(S)}$. Now swap one item: replace $j \in S$ by some $m \notin S$, giving S' . The size does not change, so the constant $\frac{1}{n-1}$ is the same on both sides. Every item other than j and m contributes the same b to both and cancels; every pair not involving j or m likewise cancels. What is left is the pairs j had with the rest, removed, and the pairs m gains with that same rest:

$$E(S') - E(S) = (b_m - b_j) + \alpha_m^\top \bar{\alpha}_j(S) - \alpha_j^\top \bar{\alpha}_j(S) = u_m - u_j. \quad (20)$$

That is exactly the difference Eq. 12 scores when choosing between m and j . So the model’s answer to “what goes in this slot, given the rest of the cart” is the conditional of $P(S) \propto e^{E(S)}$ at fixed basket size — which is what makes the Gibbs sampler of Section 6 legitimate, and is checked numerically in Section 8.2.

4.5 The probability of a whole basket

Multiplying the four answers together gives what the model says about a trip. Write

$$\mathcal{B} = \left(\underbrace{y_1 \dots y_C}_{\text{which categories}}, \quad \underbrace{(k_c)}_{\text{how many products each}}, \quad \underbrace{(J_c)}_{\text{which products}}, \quad \underbrace{(q_j)}_{\text{how many units each}} \right) \quad (21)$$

for a basket, where J_c is the set of k_c products taken from category c . Prices, the store, the day and the household are given, never generated. Then

$$P(\mathcal{B}) = \prod_{c=1}^C \underbrace{\sigma(\eta_{ict})^{y_c} (1 - \sigma(\eta_{ict}))^{1-y_c}}_{\text{Eq. 10: bought or not}} \cdot \prod_{c: y_c=1} \left[\underbrace{P(k_c)}_{\text{Eq. 11}} \underbrace{P(J_c \mid k_c)}_{\text{Eq. 12}} \prod_{j \in J_c} \underbrace{P(q_j)}_{\text{Eq. 13}} \right]. \quad (22)$$

Every category contributes a coin-flip factor whether bought or not; a bought category contributes three more. Reading Eq. 22 left to right is exactly the sampler of Section 6, stage by stage.

4.6 What is fitted, and the three ways it differs from Eq. 22

Training does not maximise $\log P(\mathcal{B})$. It minimises a weighted sum of four losses (Section 5.1), and three differences are worth naming because they change what the reported numbers mean.

(i) *The choice term is conditional on the rest of the cart.* Eq. 12 draws k_c products with no reference to the other categories. Training instead scores each purchased row against the whole rest of the basket, through $\bar{\alpha}_j(S)$, and so maximises $\sum_j \log P(j \mid \mathcal{B} \setminus j)$. Eq. 20 shows those conditionals belong to a genuine distribution, which is what licenses the sampler; but they are not the factor $P(J_c \mid k_c)$ in Eq. 22.

(ii) *The two choice sets once differed, and no longer do.* Earlier versions trained the item head against negatives drawn from the *whole catalogue* while generating within a category, so the term was fitted as a catalogue-wide choice and used as a within-category one. Eq. 12 now normalises over the category’s stocked items in both places (Section 5.2), so this particular gap is closed. It is recorded because the co-occurrence results in Section 8.7 were once attributed to it.

(iii) *The terms are separately averaged and weighted.* Each loss is a mean over its own sample — purchase rows, sampled categories, bought categories — then multiplied by w_q, w_c, w_b . Eq. 22 implies particular relative weights; the objective does not reproduce them.

Consequence. The model has a well-defined distribution over baskets, Eq. 22, and Section 8.7 tests it by sampling from it and comparing against real baskets. It has no tractable likelihood for a whole basket: no number in this paper is $\log P(\mathcal{B})$ on held-out data. Every likelihood reported is the conditional in (i).

5 Inference

We fit by stochastic gradient descent on a weighted sum of the four terms. This section gives the objective, the three sampling schemes it depends on, and the computational cost. Code: `scripts/27_nested_basket.py`.

5.1 Objective

$$\mathcal{L} = \mathcal{L}_{\text{item}} + w_q \mathcal{L}_{\text{qty}} + w_c \mathcal{L}_{\text{inc}} + w_b \mathcal{L}_{\text{brd}} - \tau_{\text{repr}} \|\Theta_{\text{repr}}\|^2 - \tau_{\text{inc}} \|\Theta_{\text{inc}}\|^2 - \tau_{\text{price}} \|\Theta_{\text{price}}\|^2. \quad (23)$$

Three penalty blocks, not two. Θ_{repr} covers $\alpha, \theta, \eta, \mu, \delta, \zeta, \xi, w^{\text{loy}}, w^{\text{frq}}$; Θ_{inc} covers c^u, c^c, c^s ; Θ_{price} covers only $\gamma, \beta, \gamma^q, \beta^q$, at $10^{-4}/1500$ against $10^{-2}/1500$ for the representation block. Shrinking the price block at the representation block’s rate biases the elasticity toward zero, and the elasticity is the estimand.

Θ_{inc} has to be separate, and this is not a tuning preference. Fitted inside Θ_{repr} under a single τ , the incidence embeddings collapse to near rank one: 144,256 parameters delivering roughly one number per household times one per category, which cannot express “buys nappies, never cat food”. Averaged over all dimensions the data gradient beats the penalty 5.8–17 $\times$, but that average is carried by the one direction that is learning; on the rest the data gradient is near zero and the penalty is the only force. Separating the block raises the entropy effective rank of c^u from 5.2 to 31.8 and of c^c from 4.1 to 23.7, and improves incidence NLL $0.1249 \rightarrow 0.1199$ against a 0.1132 oracle floor.

A discrepancy worth stating. Eq. 23 is not quite what the code maximises. The $C/|\mathcal{S}_v|$ rescaling that makes $\mathcal{L}_{\text{inc}}$ an unbiased estimate of the full 188-category likelihood means that at $w_c = 1$ the incidence term carries $M \cdot C \approx 36,096$ summands against the item term’s $\approx 1,476$; the implementation instead takes a plain mean of each component over its own term count, which weights incidence about 24.5 $\times$ lower. Fitting Eq. 23 literally ($w_c = 24.46$) buys 0.0035 nats of incidence likelihood and costs **63% of the price coefficient**, $+0.695 \rightarrow +0.260$ — roughly 42

refit standard deviations — because γ, β reach incidence only through $b \rightarrow IV$, while the item head carries 79% of the own-price elasticity. Neither weighting is *a priori* correct, since any $w \neq 1$ forfeits the pseudo-likelihood consistency argument; every number in this paper comes from the mean-over-terms convention.

Optimiser Adam, learning rate 0.005 annealed by cosine to $0.05 \times$ over 6,000 iterations. A batch is 192 baskets expanded to all their purchase rows. Validation runs every 1,000 iterations and the checkpoint is kept on the sum of all four held-out terms; because the item conditional is exact, that criterion needs no surrogate and no calibration.

5.2 There is no negative sampling

Earlier versions of this model sampled K negatives from $p(j) \propto \max(\text{count}_{\text{train}}(j), 1)^{0.75}$ over the whole catalogue and scored a softmax over $\{\text{chosen}\} \cup \{\text{negatives}\}$. That is no longer done anywhere, and the reason is structural rather than computational.

The conditional the model actually defines normalises over A_{-j} , the purchased item’s own category minus whatever else the basket holds from it — a median of 15 products and at most 225. That sum is affordable *exactly*. So the item head evaluates the true conditional, and with it go the proposal distribution, the $\log q$ correction, and any metric whose value depends on how decoys were drawn.

This matters for reading the literature as much as for the fit. A catalogue-wide decoy is almost always separable by $\lambda_j + \theta_i^\top \alpha_j$ alone, so a sampled loss asks “is this item plausible at all” rather than “does this item fit *this* basket”, and its optimal scorer is $\log p - \log q$ rather than $\log p$. Item log-likelihoods reported against sampled decoys — including those in earlier versions of this paper — are not comparable to the numbers in Section 8; only distance from chance carries across.

5.3 The inclusive value is estimated, not summed

IV_{ict} is a log-sum-exp over a category’s stocked items. Categories are padded to the largest (225 items) although the median has 15, so a dense evaluation spends most of its work on padding. Write n_c for the number of items of category c the trip’s store carries. We sample $m = 32$ of them uniformly with replacement and rescale:

$$\widehat{IV} = \log \left(\sum_{s=1}^m e^{u_{js}} \right) + \log \frac{n_c}{m}. \quad (24)$$

The *sum* inside is unbiased, since $\mathbb{E}[\frac{n_c}{m} \sum_s e^{u_{js}}] = \frac{n_c}{m} \cdot m \cdot \frac{1}{n_c} \sum_j e^{u_j} = \sum_j e^{u_j}$. Its *logarithm* is not: by Jensen $\mathbb{E}[\log X] \leq \log \mathbb{E}[X]$, so $\widehat{IV}$ is biased slightly low. The bias is absorbed into the category intercept c_c^0 during training, and generation uses the same estimator, so the two remain consistent.

A frozen reference. $\overline{IV}_c$ in Eq. 10 is fixed once, at iteration 500, to the mean IV over a *uniform* category sample. Any fixed constant is absorbed by c_c^0 , so freezing does not change the fit; it guarantees that training and generation subtract the same thing. Taking the reference from the training batches instead uses whatever category mix those batches had, and if that mix differs from the population the generated baskets come out systematically wrong.

5.4 Category sampling for the incidence head

Incidence is a Bernoulli over 188 categories per trip, of which only 6.12 are bought. We sample 16 categories per trip **uniformly**. Case-control sampling (a few bought, a few not) is tempting

at a 3.25% base rate, and biases the fit two ways: the intercept, which the standard $\log(\pi_1/\pi_0)$ offset corrects, and the term $\kappa_c(IV - \overline{IV}_c)$, whose mean differs between sample and population because bought categories have higher inclusive values — which the offset does not touch. Uniform sampling removes both at source and needs no correction.

5.5 Computational cost

run	iterations	wall time	ms/iter
default	12,000	10.8 min	53.8

Two lookups dominate and both are sparse and high-cardinality: household state and store price. Materialising (household $\times$ day $\times$ sub-commodity) is about 32 million rows and a per-sample dictionary is about 35 million hits per epoch. Both are instead stored once as a globally sorted key array with key = group $\cdot$ stride + index, so a whole batch resolves in one `np.searchsorted`; the stride exceeds any index so ordering never crosses a group boundary. That search is still roughly 40% of a step, and is avoidable by deduplicating repeated candidates within a batch, which we have not done.

Scalability. Cost is independent of catalogue size (negative sampling fixes the candidate count), of the number of households (only the embedding table grows), and of the number of baskets per step. Categories are capped at 16 sampled per trip and items per category at 32 by the sampled inclusive value. Nothing is quadratic in items or households.

6 Sampling

Generation is a harder requirement than prediction and is where most of this paper’s findings came from. We state precisely what the model gives us and what it does not.

What we have. Conditionals: $P(\text{item} \mid \text{rest of basket})$ from the item head, $P(\text{category bought})$ from the incidence head, and count distributions from the quantity and breadth heads. **What we do not have** is a normalised joint $P(S)$: computing it needs a partition function over all size- n baskets, which is intractable. Eq. 20 shows the item head’s conditionals are those of $P(S) \propto e^{E(S)}$ at fixed n , so we can *sample* that joint without ever evaluating it.

Stage 4 holds basket size and category composition fixed, which is the move Eq. 20 is defined over. Stage 1’s optional sweep is the same argument one level up: the category indicator is a C -dimensional binary field whose single-site conditionals the incidence head supplies.

A sequential alternative. If the model is trained with a random-prefix context — one permutation per basket, each row seeing only the items before it, which is SHOPPER’s unordered likelihood (its Eq. 6) — then Stages 3–4 collapse into one pass that places items one at a time with $\bar{\alpha}$ the running mean of what is already placed. Training and generation then use identical conditionals and no Gibbs is needed. This is implemented behind a flag and is not evaluated in this paper.

What generation does not carry. With the context zeroed at Stage 3 and no Stage 4, generated baskets reproduce marginals only. This is not a detail: it is what Section 8.7 measures.

Algorithm 1: Basket generation

Input: household i , day t , week w , store s **Output:** a basket of items with unit counts**Stage 1 — categories.** For each c , compute IV_{ict} and draw $\mathbb{I}\{c \in B\} \sim \text{Bernoulli}(\sigma(\text{logit}_{ict}))$;**if** *the category interaction is on* **then**| Gibbs sweep the C -dimensional indicator: for each c , recompute logit_{ict} given the current $B \setminus c$ and redraw;**end****Stage 2 — how many.** For each $c \in B$ draw $k_c = 1 + \text{Poisson}(e^{z_{ic}^b})$, capped at the number the store stocks;**Stage 3 — which.** For each $c \in B$, draw k_c distinct items from $\text{softmax}_j u_{ijt}$ over that category’s stocked items, with context zero (no basket exists yet);**Stage 4 — refine.** Gibbs sweeps: for each filled slot, remove its item, recompute $\bar{\alpha}$ from the current draft, and redraw from that slot’s own category;**Stage 5 — units.** For each chosen item draw $1 + \text{Poisson}(e^{z_{ijt}})$;

7 From measurement to model

what Section 3 showed	what the model does
25% of the units response is existing buyers taking more	quantity term with its own price coefficient
73% of the units response is entering a category at all	the nest, with κ_c estimated rather than assumed
1.284 distinct items per purchased category; 22.3% of rows buy more than one unit	breadth term; units kept as counts, never binarised
Taste is $1.66\times$ more self-similar within household than across	per-household θ_i
Per-household price slopes have split-half reliability +0.236	price sensitivity low-rank $\gamma_i^\top \beta_j$, not free per pair
Median store carries only 63% of the catalogue	availability mask: unstocked items leave the choice set
15.8% of store-item-weeks differ from the chain price	store-level price deviation where observed
Week effects carry a confound between price and demand	$\mu_j^\top \delta_w$ seasonality control
Promotion weeks are $3.4\times$ steeper than ordinary variation	<i>not addressed</i> ; Section 10

8 Empirical study

Every number in this section comes from one fitted model. That sentence is doing work: an earlier version of this section mixed results from two fits without naming either, and two of its conclusions did not survive being recomputed on a single one.

8.1 Protocol, and what is comparable

All numbers are on held-out test weeks. The item score is the exact conditional — a softmax over the purchased item’s category minus whatever else the basket holds from it — so it depends on no sampling scheme. The choice set averages **46.9** products, making chance $-\log 46.9 = -3.848$ and 2.1% top-1.

Two sources of uncertainty are separated because they cover different things. **Refit variability** comes from seven independent fits differing only in seed: item sd **0.0040**, top-1 sd 0.0013, price coefficient sd 0.0104, κ sd **0.0011**. A gap between two independently fitted models must exceed **0.0110** nats to clear it at 95%. An eighth fit at a seed outside the seven landed at item -2.5024 , price $+0.7085$, κ 0.8701 — inside the distribution on all three, which is the only out-of-sample check these standard deviations have. **Evaluation sampling** is a household block bootstrap: 400 draws over 1,664 households and 6,000 baskets, paired so the interval is on the difference. Neither covers resampling the training households, which would need a refit per replicate.

8.2 Do the equations match the code?

Thirty checks compare each printed formula against the fitted model — by autograd where this paper states a derivative, by direct evaluation where it states an identity, and by rebuilding the quantity from the raw training split where it states a definition (`eval/33_verify_equations.py`). All thirty pass, on every model fitted for this paper.

They did not always. An earlier version of this paper claimed fifteen checks passing while the suite could not run at all: its label argument was never read, so it silently loaded a superseded checkpoint. Behind that, its quantity check still indexed candidate slot 0 after the loss was fixed to index the purchased item, so it had been *failing*; and its term-by-term check for Eq. 7 omitted two terms and passed anyway, because the pair it tested happened to have no purchase history. Three divergences between this paper and the code surfaced once the suite ran — the persistence terms of Eq. 9, the habit loading in the incidence predictor, and the third penalty block of Eq. 23 — and all three are now stated here because the code has always had them.

8.3 Ablations

Eight fits, differing only in which term is removed, all 6,000 iterations at one seed with identical data and schedule.

removing	item log-lik	cost	clears refit noise?
— (full model)	−2.5118	—	—
quantity head	−2.5113	−0.0005	no , and correctly so
nest	−2.5334	+0.0215	yes, by 2×
household state	−2.5373	+0.0255	yes
prices scrambled	−2.5706	+0.0587	yes
basket interaction	−2.5818	+0.0700	yes
persistence	−2.5833	+0.0714	yes
store	−2.8240	+0.3122	yes

With bootstrap intervals on the paired gaps: no persistence $+0.0728$ [$+0.0648, +0.0816$]; the shared incidence penalty $+0.0715$ [$+0.0635, +0.0804$]; prices scrambled $+0.0562$ [$+0.0489, +0.0626$]. Every interval excludes zero.

Store dominates at 0.312, nearly four times the next term. Removing the quantity head costs -0.0005 — inside refit noise and marginally the wrong way, which is what a null looks like, and the only evidence here that the procedure can return one. κ is stable at 0.865–0.872 across every variant including the placebo, and sits below 1 throughout.

Two terms that look independent are substitutes. Before the persistence terms of Eq. 9 were added, the household state was worth +0.049 and the basket interaction +0.058. State has now halved to +0.026 while the interaction grew to +0.070. ω_{ij} and f_{ij} are per-(household, item) purchase history, which overlaps with what x_{ijt} carried: persistence did not simply add signal, it absorbed about half of the state term’s job.

Removing the nest is the only ablation that lowers the price coefficient (+0.695 $\rightarrow$ +0.596). With no $\kappa \cdot IV$ channel, price cannot reach category entry at all, and the fit shrinks the coefficient rather than redistributing it. Every other ablation nudges it up, toward +0.71–0.75.

8.4 Against learned baselines

HPF and B-Emb are refitted and scored on identical choice sets. They bracket the model deliberately: HPF has the household half and no basket interaction, B-Emb the basket half and no household.

model	log-lik	top-1
HPF, Poisson factorisation	−3.1866	0.165
B-Emb, skip-gram embeddings	−3.0939	0.187
full model, no persistence	−2.5787	0.322
full model	−2.5080	0.341

The gap over the stronger baseline is +0.5860 nats, 95% CI [+0.5628, +0.6094] by paired household bootstrap.

SHOPPER, on both metrics. SHOPPER ships as CUDA C++ and could not be run on the available hardware, so it is reimplemented in PyTorch from the paper and its source: untied ρ , sequential prefix context, catalogue-wide softmax. Its choice set is the whole catalogue while ours is a category, so scoring only on ours would flatter us; both are therefore scored on both, exactly. Within category SHOPPER reaches -2.8262 against our **-2.5058** ; on the full catalogue -7.0501 against **-6.7402** . The margin of +0.310 nats survives on SHOPPER’s own normalisation, which is the comparison that matters.

This is a floor, not a fair fight. Same metric and choice sets, but one tuned scalar per baseline against 881,490 parameters and an architecture selected over many runs. SHOPPER is also Bayesian and maximises an ELBO while this fits the same model by MAP, so the comparison is between models, not inference procedures, and the SHOPPER column carries no uncertainty of its own.

8.5 Is the price response identified?

Refitting on a price panel scrambled before fitting, store deviations included:

model	price coef.	allocation elasticity	κ
fitted	+0.695	-0.7171 [-0.7646, -0.6713]	0.8723
prices scrambled	+0.000	+0.0000 [-0.0029, +0.0000]	0.8720

The placebo retains 0.0% of the coefficient on both margins — the fitted value is 5×10^{-39} — while κ moves by 0.0003 and the incidence likelihood is untouched. Prices scramble; category structure does not.

What this does and does not show. Permuting a product’s price series across days destroys the price–demand association *whether it is causal or confounded*. It rules out a spurious constant. It cannot distinguish “price moves demand” from “the retailer times promotions to demand”, which is the actual endogeneity concern for within-product price variation. There is no instrument here and no discontinuity, which is why this subsection is no longer titled “is the price response causal?” — the design does not answer that question.

8.6 Where a price cut goes

Own-price elasticity -1.2070 over 12,724 (household, item) pairs: allocation -0.9562 (79%), incidence -0.1393 (12%), quantity -0.1115 (9%). Mostly a price cut moves share. The quantity margin is unreachable for a binary-purchase model and is the concrete return on modelling units at all. A fourth margin exists through breadth, via $-c_c^p/N_c$ in the breadth rate, and has not been measured. Median κ is 0.872 and *no* category exceeds 1: a price cut reaches category entry at less than full strength everywhere.

8.7 Generation

Generated on 24,768 held-out trips with the same household, day, week and store.

	real	generated	TVD
items per basket	8.18	7.47	0.282
categories per basket	6.38	5.60	0.292
units per basket	11.06	10.23	0.272
distinct items per category	1.28	1.33	0.066

The generator makes baskets too small — 9% short on items and 12% on categories. On the superseded fit this ran the other way, so it is a defect the current model introduced rather than an inherited one, and it is unexplained. Item marginals still correlate +0.872 across the catalogue with 0.4% of real products never generated, and the price response re-estimated from generated data recovers 74–76% of the real elasticity. That last check is circular — it re-estimates a quantity the model was fitted to, so it verifies that the sampler propagates the coefficient, not that the coefficient is right.

Co-occurrence lift correlates with real at Spearman +0.099 after four Gibbs sweeps, with mean lift 3.23 real against 1.82 generated: the right ordering, weakly, and far too little concentration. Within-basket pairs sharing a held-out sub-commodity label run 0.0482 generated against 0.0646 real, so fine substitution is 25% under-produced; at department level it is slightly *over*-produced, so the failure is specific to fine distinctions.

A joint test. Every likelihood above is a conditional, and comparing summary statistics one at a time cannot detect a joint mismatch. A classifier two-sample test can: a discriminator separates real from generated baskets at **AUC 0.7725** on eight basket-level features. Its two most informative features are category count and basket size — the two marginals that are off — so the joint failure is not yet separable from the marginal one.

Novelty, and a conclusion that reversed. Generated baskets place a never-before-bought item 19.2% of the time against a real 14.2%. An earlier version of this paper put that figure at 46% and concluded, from a temperature sweep that barely moved it, that the item conditional’s sharpness was not the cause. Both halves were artefacts of a measurement that attributed each generated basket to the wrong household. Measured correctly, sharpening the generation softmax to $T = 0.70$ takes novelty *past* the real rate, to 11.3%. Sharpness is a live lever; what disqualifies it as a fix is that scaling the item utility also scales the inclusive value, so basket size inflates from 7.55 to 12.39 items on the way.

Rollouts drift. The excess compounds: cumulative distinct new items run $1.156\times$ real after one trip and $1.261\times$ after twelve. That is a lower bound, since generation reads the recency state from the real history, so a generated novel item does not make the next basket more novel. Single-step counterfactuals are supported; multi-step rollouts are usable over short horizons at most.

Two negative results. Contrastive divergence with a persistent pool, which trains directly against the model’s own samples, moved the novelty rate by nothing at either weight tested, while costing 0.03–0.04 nats. So did an explicit habit feature over household and category, which is nonetheless still in the fitted model, at +0.006 nats. Both were re-measured after the attribution bug was corrected, and both nulls stand.

8.8 What the representations learned

Products. α recovers sub-commodity structure at **$27.8\times$** chance on 10-nearest-neighbour purity — 0.1126 [0.1041, 0.1260] against a permutation null at 0.0039, a random embedding of identical shape at 0.0029, and chance at 0.0040. Both nulls sit at chance, so the metric is not high for structural reasons. The lift *grows* as labels get finer: $1.3\times$ at department, $25.6\times$ at category, $73.7\times$ at sub-commodity. But all-pairs AUC is at or below chance for the two coarse levels (0.496, 0.463) and the silhouette is negative throughout. Restricting to pairs co-bought at least eight times, cosine separates same-sub-commodity pairs at AUC 0.928 with mean cosine +0.619 against +0.138. α is organised where the data is dense and unstructured where it is sparse.

Households — a negative result that survives a non-linear probe. Linear *and* gradient-boosted probes on θ , c^u and γ fail to beat the majority class on income, household size, age, children or home ownership; the best boosted probe comes in 0.037 *below* it, and on income the shuffled-label control outscores the real labels. The household parameters encode purchase behaviour, not who the household is. At 655 households this cannot rule out a relationship a larger sample would reveal.

Price sensitivity. $g_{ij} = \gamma_i^\top \beta_j$ has 46.8% of its variance between products and 17.6% between households: price sensitivity is mostly a property of the product, which is worth stating plainly because the household-specific slope is the model’s main claim to heterogeneity. The category

ordering was never supplied and is economically sensible — least elastic CIGARETTES +0.104 and BABY FOODS +0.127, most elastic BATH TISSUES +2.002 and COLD CEREAL +1.816. The same ten categories appear in the same two groups on the superseded fit, so this is the most robust evidence here that $\gamma^\top \beta$ estimates something real. Household price sensitivity is unrelated to income (Spearman +0.005, $p = 0.89$), and 24.0% of pairs carry the economically wrong sign, which nothing in the specification forbids.

Coupon targeting — a screen, not a score. Ranking households by predicted g_{ij} does *not* work: Spearman against the real held-out slope is -0.122 , and the quintile ordering breaks badly, with the second quintile showing a real slope of $+0.141$. An earlier version of this paper reported a $22\times$ difference between the top and bottom quintile pairs; on a single model the bottom two average $+0.061$, the wrong sign, and the ratio is meaningless. What survives is the sign: pairs with $g > 0$ show a real slope of -0.187 against -0.004 for the 21% with $g \leq 0$. Used as a binary screen the coefficient is informative; used as a graded score it is not.

9 Related work

SHOPPER [1] models a basket as a sequence of choices with an asymmetric interaction $\rho_c^\top \bar{\alpha}$, per-customer price sensitivity as a low-rank bilinear form, and week effects for seasonality. We inherit all three. We differ in adding an explicit incidence stage with an estimated nesting coefficient, in modelling unit counts rather than binary purchase, and in conditioning on the store; and we replace its variational inference with maximum *a posteriori* estimation by SGD, which loses uncertainty quantification and gains an order of magnitude in fitting time. Its asymmetric interaction was tried and did not improve generation on this dataset, and its group-biased negative sampling is moot here, since the item conditional is now summed exactly (Section 5.2).

Section 8.4 compares against the two latent-factor baselines SHOPPER uses — hierarchical Poisson factorization [3] and exponential family embeddings [4] — and adopts its evaluation on high price-variation purchases.

The nested factorization of [2] supplies the nesting coefficient’s interpretation and the argument that week effects suffice to identify the price effect. Its within-category softmax requires at most one item per category per trip, which 68.2% of baskets here violate.

10 Discussion

What the model does well. It separates three margins that are economically distinct and estimates their coupling; its price response survives a structural placebo; and synthetic baskets carry 87% of the real price elasticity, which is the property that matters if the generator is to be used for policy learning.

What it does not do.

1. **Per-pair co-occurrence is only partly learned.** At best the per-pair rank correlation is $+0.091$ — real ($z = +14.1$) but far from strong. Generated baskets have roughly the right amount of coherence and only weakly the right pattern.
2. **Promotion weeks are steeper than a single price coefficient allows.** Section 3 measures an implied -2.83 in the cut week against -0.792 on ordinary variation. Display and feature are not observed, so one coefficient averages two regimes.

3. **Basket-size distributions are wrong in shape** even where means agree.
4. **There is no held-out joint likelihood.** The model has a well-defined generative distribution over baskets (Eq. 22) but no tractable likelihood for one: the item stage is fitted as a conditional, its softmax is normalised over a different set than the generative one, and the four terms are weighted rather than multiplied. Every likelihood in this paper is the *conditional* of Section 4.5, never $\log P(\mathcal{B})$. Generation, evaluated in Section 8.7, is the only test of the joint.
5. **The generator makes baskets too small**, 9% short on items and 12% on categories (Section 8.7). The earlier fit erred in the opposite direction, so this is a regression the current model introduced, and we have not explained it.
6. κ **is weakly identified**, having moved with the incidence sampler rather than the data before settling.
7. **Partial baskets need a correction at serving time.** The interaction term is trained on a context averaged over a whole basket, and a one-item cart produces a vector $2.31\times$ that norm, so without rescaling a cart with one item scores worse than an empty one. Measured on an earlier fit and not re-measured here.
8. **No uncertainty quantification.** We report point estimates; the seed spread bounds run-to-run noise but is not a posterior.

A methodological lesson. The most useful finding in this paper was not a model. Eleven changes to the interaction term — including the one a well-supported diagnosis predicted would work — moved the target quantity by nothing, and a change to the negative sampling distribution moved it by fourteen standard errors. The distinguishing evidence was a dose-response curve, and none of it would have been visible without a model-free distributional evaluation of the generator. Where a model is meant to generate data, it should be judged on the data it generates.

References

- [1] F. J. R. Ruiz, S. Athey and D. M. Blei. *SHOPPER: A Probabilistic Model of Consumer Choice with Substitutes and Complements*. Annals of Applied Statistics, 2020. arXiv:1711.03560.
- [2] S. Donnelly, F. J. R. Ruiz, D. M. Blei and S. Athey. *Counterfactual Inference for Consumer Choice Across Many Product Categories*. Quantitative Marketing and Economics, 2021.
- [3] P. Gopalan, J. M. Hofman and D. M. Blei. *Scalable Recommendation with Hierarchical Poisson Factorization*. UAI, 2015.
- [4] M. Rudolph, F. J. R. Ruiz, S. Mandt and D. M. Blei. *Exponential Family Embeddings*. NeurIPS, 2016.
- [5] M. K. Titsias. *One-vs-Each Approximation to Softmax for Large Scale Classification*. NeurIPS, 2016.